GRAY SENTINEL - WEEK 2 CAPSTONE

Cloud-Native APT Investigation

Multi-cloud security investigation across AWS, Microsoft Azure, Google Cloud Platform, Kubernetes, and the Elastic Stack.

Report type	Security Lab / Capstone Investigation Report
Focus	Cloud enumeration, misconfiguration analysis, centralized logging, Kubernetes security audit, automation, and evidence packaging
Environment	Authorized GraySentinel training/lab environment

This report is based on the supplied terminal evidence. Identifiers and entities shown in the lab may be simulated or sanitized.

1. Executive Summary

This capstone demonstrates an end-to-end investigation workflow for a simulated Advanced Persistent Threat (APT) scenario spanning AWS, Azure, GCP, and Kubernetes. The exercise combines cloud asset discovery, identity enumeration, network-security review, centralized logging, Kubernetes benchmark validation, security scanning, automated evidence collection, and integrity verification.

Outcome: The supplied evidence shows completion of the principal lab phases, including cloud CLI setup, AWS/Azure/GCP enumeration, Elastic Stack startup, Logstash ingestion pipelines, automated investigation scripting, kube-bench and kube-hunter execution, report generation, artifact packaging, and SHA256 hashing.

The lab's own investigation summary identifies representative cloud-security issues such as public storage exposure, permissive IAM or service-principal permissions, privileged accounts without sufficient protection, exposed Kubernetes services, and weak pod security controls. These findings should be interpreted as training-scenario observations rather than claims about a production environment.

2. Scope and Objectives

• Prepare AWS, Azure, and GCP command-line tooling on Kali Linux.

• Enumerate cloud assets, subscriptions/projects, users, identities, compute resources, storage, and network controls.

• Deploy Elasticsearch, Logstash, and Kibana for centralized security telemetry.

• Configure log ingestion paths for AWS CloudTrail, Azure Event Hubs, and GCP Pub/Sub.

• Automate multi-cloud evidence collection with Bash.

• Assess Kubernetes configuration against CIS guidance using kube-bench.

• Identify Kubernetes attack-surface weaknesses using kube-hunter.

• Package collected artifacts and generate a SHA256 integrity hash.

3. Tools and Technologies

Area	Tools	Purpose
Cloud	AWS CLI, Azure CLI, Google Cloud CLI	Authentication, asset discovery, identity and resource enumeration
Logging / SIEM	Elasticsearch, Logstash, Kibana, Docker Compose	Centralized ingestion, indexing, search, and visualization
Kubernetes	kube-bench, kube-hunter	CIS configuration audit and attack-surface/security scanning
Automation	Bash	Repeatable multi-cloud evidence collection
Integrity	tar, sha256sum	Artifact packaging and integrity verification

4. Investigation Methodology

```
Phase 1 - Environment Setup
        |
Phase 2 - Multi-Cloud Reconnaissance / Enumeration
        |
Phase 3 - Centralized Logging with Elastic Stack
        |
Phase 4 - Kubernetes Security Audit
        |
Phase 5 - Automated Investigation and Report Generation
        |
Phase 6 - Packaging, Hashing, and Submission Evidence
```

The workflow follows a defensive incident-response approach: first establish visibility, then collect cloud inventory and identity context, centralize logs for correlation, validate Kubernetes security posture, automate repeatable collection, and finally preserve artifacts with an integrity hash.

5. Phase 1 - Environment Preparation

The lab begins with package index updates and installation or verification of the three primary cloud CLIs. AWS CLI is installed through APT, Azure CLI through the Microsoft installation script, and Google Cloud CLI through the Google package repository.

> **Evidence note:** One Google package-key command in the supplied terminal evidence returns the curl usage screen, indicating that particular command did not execute as intended. However, later gcloud --version output confirms that Google Cloud CLI was available in the lab environment.

6. Phase 2 - Multi-Cloud Reconnaissance

AWS

AWS discovery uses aws s3 ls, aws s3api list-buckets, and aws ec2 describe-security-groups. The evidence shows five lab S3 buckets and a security-group rule containing 0.0.0.0/0, which represents access from any IPv4 address when applied to an inbound rule.

Azure

The Azure phase shows service-principal style login syntax, subscription enumeration, Microsoft Entra ID user listing, and service-principal enumeration. These steps establish the accessible tenant/subscription context and help identify high-value identities or application principals.

GCP

GCP enumeration covers authenticated accounts, projects, Compute Engine instances, and Cloud Storage buckets. The evidence includes production, development, and staging-style projects and examples of VMs with external IP addresses.

7. Phase 3 - Centralized Logging

Docker Compose is used to start Elasticsearch, Kibana, and Logstash. Logstash then acts as a collection and forwarding layer for cloud telemetry.

```
AWS CloudTrail in S3 --------\
Azure Activity via Event Hubs ---> Logstash ---> Elasticsearch ---> Kibana / Analyst Search
GCP Logging via Pub/Sub ---------/
```

A Logstash message indicating that a pipeline started successfully confirms the pipeline configuration was accepted and started; it does not, by itself, prove that every upstream source delivered events or that all events were indexed successfully.

8. Phase 4 - Kubernetes Security Audit

kube-bench evaluates Kubernetes configuration against CIS Benchmark recommendations. It is primarily a configuration-compliance tool and reports results such as PASS, FAIL, or WARN.

kube-hunter focuses on attack-surface and weakness discovery. Used together, the two tools provide complementary coverage: configuration hardening plus externally observable security weaknesses.

9. Phase 5 - Automated Investigation

A Bash script named graysentinel-apt-investigation.sh is created to collect selected evidence from AWS, Azure, and GCP into a timestamped investigation directory. The evidence shows the script made executable and then run successfully through its three cloud phases.

Automation improves repeatability, reduces manual collection errors, and creates a structured evidence set for later analysis and reporting.

10. Evidence-Based Findings

Finding	Environment	Security Meaning	Priority
Security-group rule with 0.0.0.0/0	AWS	Potentially unrestricted inbound exposure depending on protocol/port and attachment context.	High
Public storage bucket referenced in investigation summary	AWS	Public object storage can expose sensitive data if not intentionally designed for public distribution.	High
High-privilege IAM user referenced in summary	AWS	Broad privileges increase blast radius if the identity is compromised.	High
Global administrator without MFA referenced in summary	Azure	Privileged identity without strong authentication materially raises account-takeover risk.	Critical
Over-privileged service principal referenced in summary	Azure	Application identities with excessive permissions can enable persistent or automated abuse.	High
Public storage / permissive IAM referenced in summary	GCP	Public access or broad IAM bindings can expose cloud data and services.	High
Kubelet API exposed on port 10250 referenced in summary	Kubernetes	Exposed kubelet interfaces can become a high-value attack surface if authentication/authorization is weak.	High
Pod security controls disabled referenced in summary	Kubernetes	Weak workload restrictions can permit dangerous container privileges or configurations.	High

Priority ratings are analytical interpretations for the training scenario and depend on the exact permissions, network reachability, workload sensitivity, and compensating controls.

11. Attack-Chain Interpretation

The report artifact shown in the lab references a simplified chain of **S3 Leak -> Kubernetes Breakout -> Azure AD Compromise**. A defensible interpretation of that scenario is:

```
Cloud Storage Exposure
        |
Credential / Configuration Discovery
        |
Kubernetes Access or Workload Abuse
        |
Privilege Expansion / Persistence
        |
Cross-Cloud Identity Abuse
        |
Sensitive Resource Access / Exfiltration
```

This illustrates why identity, storage, containers, and cloud activity logs should be investigated together rather than as isolated systems.

12. Recommendations

1. Remove unnecessary public storage access and enforce bucket-level least privilege.

2. Replace broad security-group sources such as 0.0.0.0/0 with trusted CIDRs wherever public exposure is not required.

3. Enforce MFA and stronger controls for privileged human identities.

4. Apply least privilege to service principals and rotate secrets or prefer managed identities where supported.

5. Review GCP IAM bindings and external IP exposure for each workload.

6. Restrict kubelet and Kubernetes control-plane interfaces to trusted networks and authenticated users.

7. Enforce modern Kubernetes Pod Security standards and minimize privileged containers.

8. Centralize cloud audit logs and create detections for privilege escalation, new credentials, public-access changes, and logging disablement.

9. Run kube-bench periodically and track unresolved FAIL/WARN findings over time.

10. Preserve investigation artifacts with cryptographic hashes and controlled access.

13. Deliverables Produced

The supplied evidence shows an investigation directory containing AWS, Azure, GCP, Kubernetes, and reporting artifacts, followed by creation of a compressed package and SHA256 hash.

```
apt-investigation-20260822/
  aws-s3-buckets.txt
  aws-iam-users.json
  azure-subscriptions.json
  azure-ad-users.json
  gcp-projects.txt
  kube-bench-report.json
  kube-hunter-report.json
  graysentinel-apt-report.md
  investigation-summary.txt

apt-investigation-complete.tar.gz
SHA256 hash generated with sha256sum
```

14. GitHub Publication

Suggested repository name
graysentinel-multicloud-apt-investigation

Suggested description
Multi-cloud APT investigation lab across AWS, Azure, GCP, Kubernetes, and Elastic Stack covering cloud enumeration, misconfiguration discovery, centralized logging, CIS benchmarking, and automated evidence collection.

Before publishing, remove or mask any real client secrets, access keys, tokens, connection strings, passwords, or organization-specific sensitive identifiers. Lab placeholders and sanitized values may remain where appropriate.

15. Conclusion

This capstone demonstrates a complete defensive workflow for investigating a cloud-native, multi-cloud threat scenario. It shows practical use of cloud CLIs, identity and resource enumeration, ELK-based log centralization, Kubernetes benchmark and security scanning, Bash automation, evidence packaging, and integrity validation. The strongest lesson is that cloud incidents frequently cross boundaries between identity, infrastructure, storage, and containers; effective incident response therefore depends on correlated visibility across all of them.

16. Evidence Gallery

The following pages preserve the supplied terminal evidence in sequence. Captions describe what each screenshot demonstrates.

Evidence 01

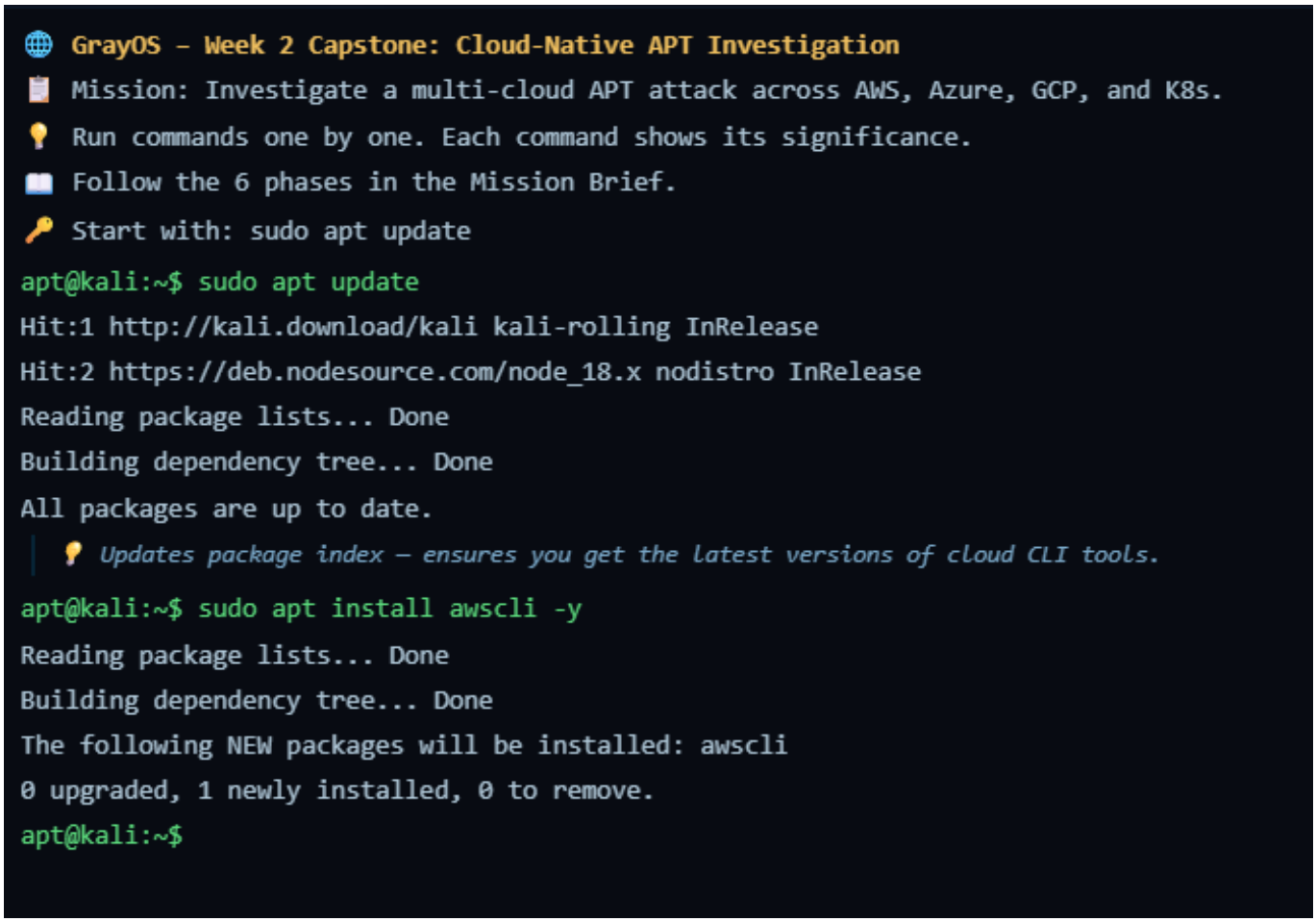


Figure 1. Environment preparation and AWS CLI installation on Kali Linux.

Evidence 02

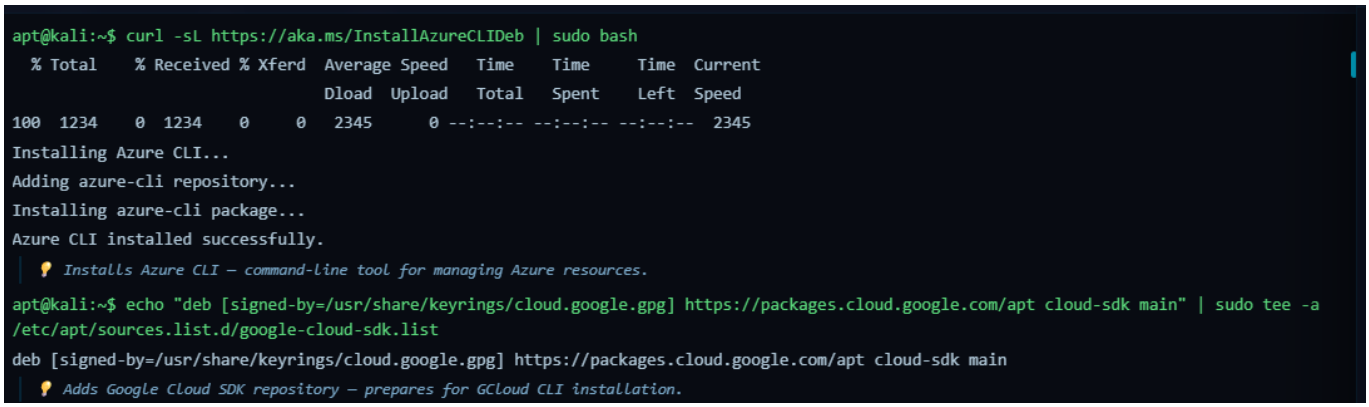


Figure 2. Azure CLI installation and Google Cloud SDK repository configuration.

Evidence 03

```
apt@kali:~$ curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key --keyring /usr/share/keyrings/cloud.google.gpg add
Usage: curl [options]
Run "help" for available commands.
apt@kali:~$ sudo apt update && sudo apt install google-cloud-sdk -y
Hit:1 http://kali.download/kali kali-rolling InRelease
Hit:2 https://deb.nodesource.com/node_18.x nodistro InRelease
Reading package lists... Done
Building dependency tree... Done
All packages are up to date.
  💡 Updates package index — ensures you get the latest versions of cloud CLI tools.
apt@kali:~$ aws --version
aws-cli/2.15.0 Python/3.11.4 Linux/6.1.0-kali3-amd64 botocore/2.4.0
  💡 Verifies AWS CLI installation and shows the version.
```

Figure 3. Google Cloud package setup attempt and AWS CLI version verification.

Evidence 04

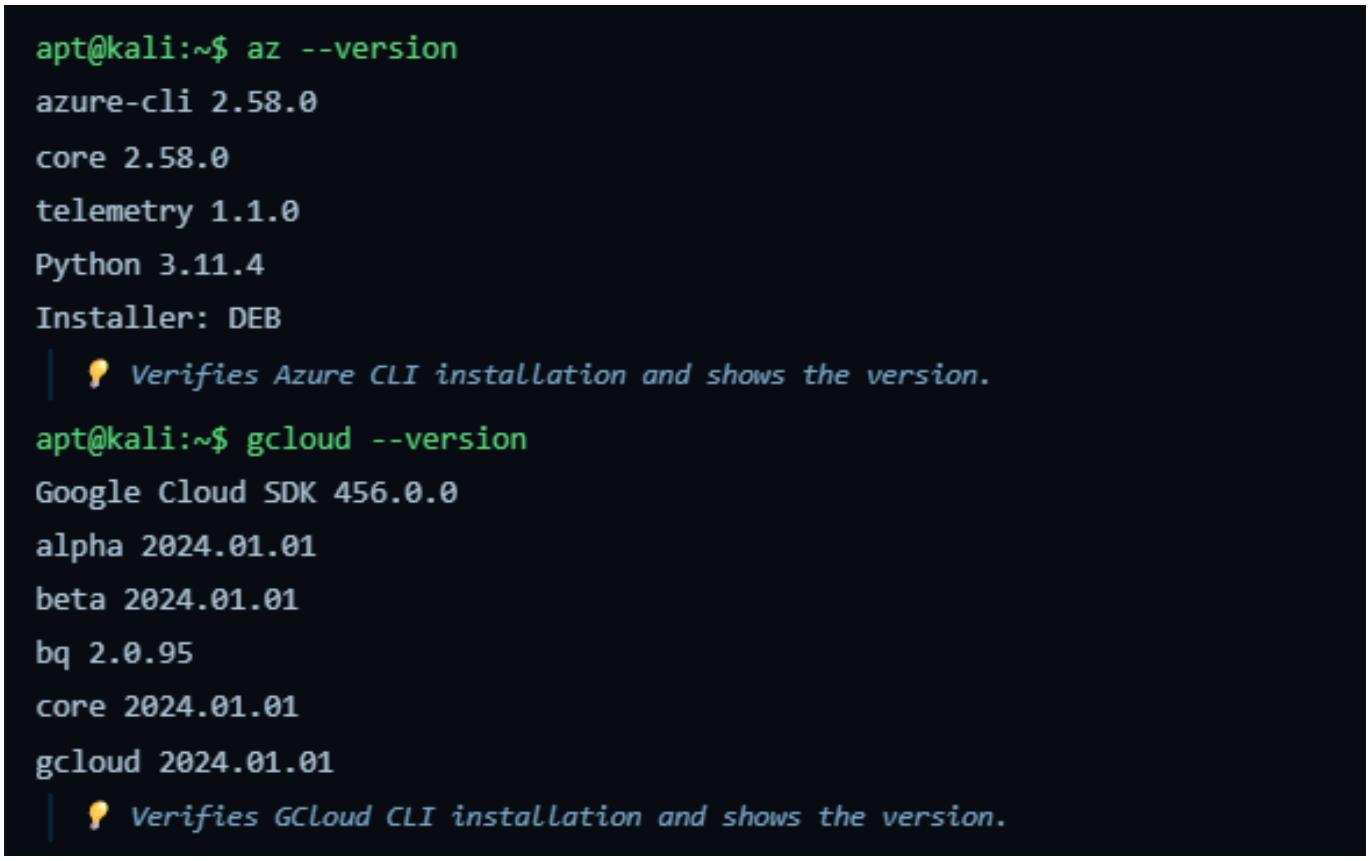


Figure 4. Azure CLI and Google Cloud CLI version verification.

Evidence 05

```
apt@kali:~$ aws s3 ls --profile graysentinel
2026-08-20 14:32:11  graysentinel-logs
2026-08-18 09:15:43  graysentinel-data
2026-08-15 22:01:27  graysentinel-backup
2026-08-10 11:45:09  graysentinel-public-assets
2026-08-05 16:20:33  graysentinel-internal
  💡 Lists all S3 buckets in the AWS account — first step in cloud asset discovery.
apt@kali:~$ aws s3api list-buckets --profile graysentinel
{
  "Buckets": [
    {"Name": "graysentinel-logs", "CreationDate": "2026-08-20T14:32:11+00:00"},
    {"Name": "graysentinel-data", "CreationDate": "2026-08-18T09:15:43+00:00"},
    {"Name": "graysentinel-backup", "CreationDate": "2026-08-15T22:01:27+00:00"},
    {"Name": "graysentinel-public-assets", "CreationDate": "2026-08-10T11:45:09+00:00"},
    {"Name": "graysentinel-internal", "CreationDate": "2026-08-05T16:20:33+00:00"}
  ]
}
```

Figure 5. AWS S3 bucket enumeration and JSON-formatted bucket inventory.

Evidence 06

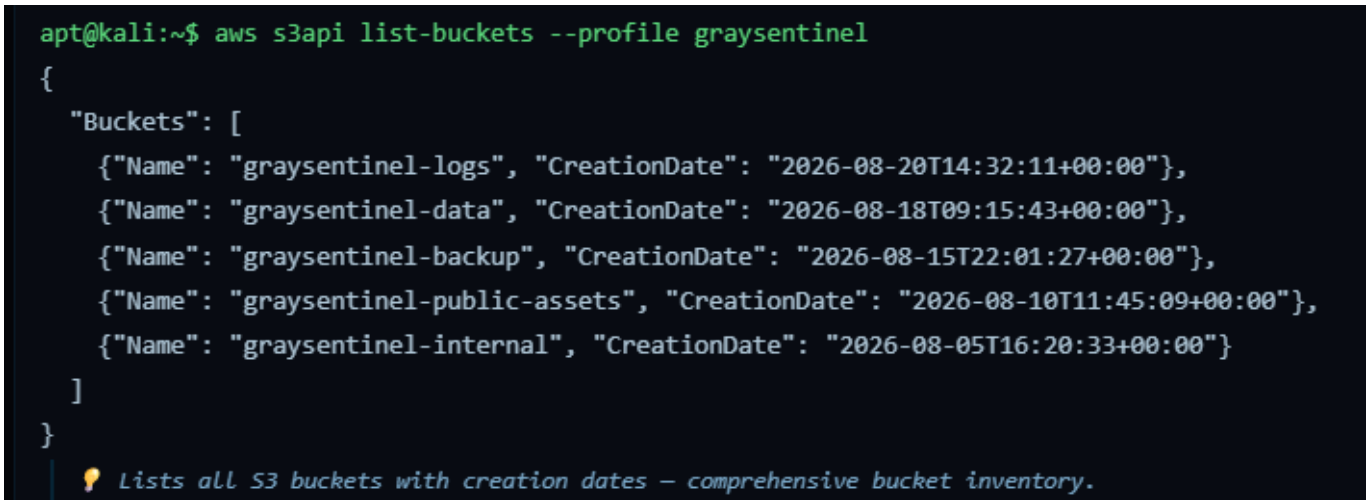


Figure 6. AWS S3 API bucket inventory with creation timestamps.

Evidence 07

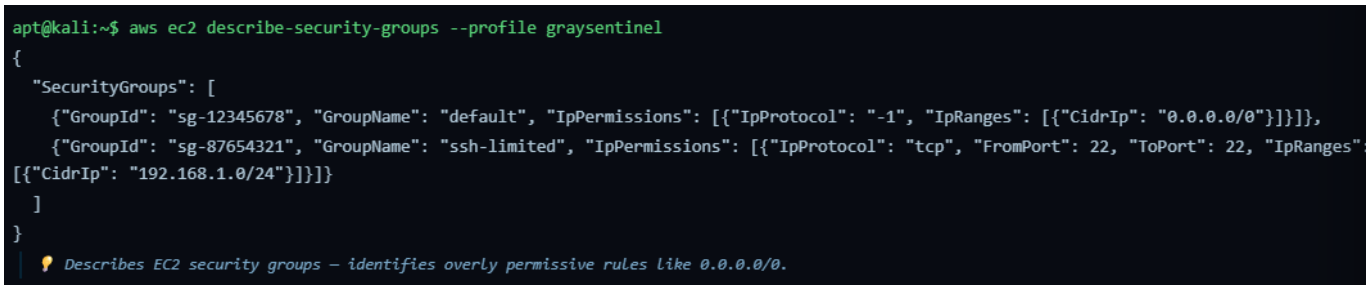


Figure 7. AWS EC2 security group enumeration, including an overly permissive 0.0.0.0/0 rule in the lab data.

Evidence 08

```
apt@kali:~$ az login --service-principal -u $CLIENT_ID -p $CLIENT_SECRET --tenant $TENANT_ID
[   {
  "cloudName": "AzureCloud",
  "homeTenantId": "12345678-1234-1234-1234-123456789abc",
  "id": "12345678-1234-1234-1234-123456789abc",
  "isDefault": true,
  "name": "GraySentinel Subscription",
  "tenantId": "12345678-1234-1234-1234-123456789abc",
  "user": {"name": "audit@target.com", "type": "user"}
} ]
  💡 Authenticates to Azure AD using service principal — enables Azure resource enumeration.
```

Figure 8. Azure service-principal authentication output showing the lab subscription context.

Evidence 09

```
apt@kali:~$ az account list --output table
Name                    SubscriptionId                        TenantId
----------------------  ------------------------------------  ------------------------------------
GraySentinel Subscription  12345678-1234-1234-1234-123456789abc  12345678-1234-1234-1234-123456789abc
Dev Test Subscription   87654321-4321-4321-4321-cba987654321  87654321-4321-4321-4321-cba987654321
  💡 Lists Azure subscriptions — shows available subscriptions and their IDs.
apt@kali:~$ az ad user list --output table
DisplayName             UserPrincipalName                     Mail
----------------------  ------------------------------------  --------------------
Admin User              admin@target.com                      admin@target.com
Audit Account           audit@target.com                      audit@target.com
Developer 1             dev1@target.com                       dev1@target.com
Guest User              guest@target.com                      guest@target.com
  💡 Lists Azure AD users — enumerates identities in the Azure tenant.
```

Figure 9. Azure subscription and Microsoft Entra ID user enumeration.

Evidence 10

```
apt@kali:~$ az ad sp list --output table
DisplayName             AppId                                 ObjectId
----------------------  ------------------------------------  ------------------------------------
Azure AD Connect        12345678-1234-1234-1234-123456789abc  12345678-1234-1234-1234-123456789abc
Office 365 Portal       87654321-4321-4321-4321-cba987654321  87654321-4321-4321-4321-cba987654321
  💡 Lists Azure AD service principals — applications with permissions in Azure.
apt@kali:~$ gcloud auth login
Your browser has been opened to visit:
    https://accounts.google.com/o/oauth2/auth?...
You are now logged in as [audit@target.com].
Your current project is [None]
  💡 Authenticates to GCP via browser — enables GCP resource enumeration.
```

Figure 10. Azure service-principal enumeration followed by GCP browser authentication.

Evidence 11

```
apt@kali:~$ gcloud auth list
Credentialed Accounts
ACTIVE  ACCOUNT
*       audit@target.com
        admin@target.com
  💡 Lists authenticated GCP accounts — shows active credentials.
apt@kali:~$ gcloud projects list
PROJECT_ID             NAME                    PROJECT_NUMBER
graysentinel-prod      GraySentinel Production  123456789012
graysentinel-dev       GraySentinel Development 123456789013
graysentinel-staging   GraySentinel Staging    123456789014
  💡 Lists GCP projects — shows all projects in the GCP organization.
```

Figure 11. GCP authenticated account listing and project enumeration.

Evidence 12

```
apt@kali:~$ gcloud projects list
PROJECT_ID             NAME                    PROJECT_NUMBER
graysentinel-prod      GraySentinel Production  123456789012
graysentinel-dev       GraySentinel Development 123456789013
graysentinel-staging   GraySentinel Staging    123456789014
  💡 Lists GCP projects — shows all projects in the GCP organization.
apt@kali:~$ gcloud compute instances list
NAME          ZONE        MACHINE_TYPE  INTERNAL_IP   EXTERNAL_IP  STATUS
web-server-1  us-central1-a  n1-standard-1  10.128.0.2     35.222.10.1   RUNNING
db-server-1   us-central1-a  n1-standard-2  10.128.0.3     35.222.10.2   RUNNING
  💡 Lists GCP Compute Engine instances — shows running VMs and their IPs.
apt@kali:~$ gcloud storage buckets list
BUCKET_NAME               LOCATION       STORAGE_CLASS  LABELS
graysentinel-gcp-logs     US             STANDARD
graysentinel-gcp-backup   US-CENTRAL1    STANDARD
  💡 Lists GCP storage buckets — shows cloud storage assets.
```

Figure 12. GCP project, Compute Engine instance, and Cloud Storage bucket enumeration.

Evidence 13

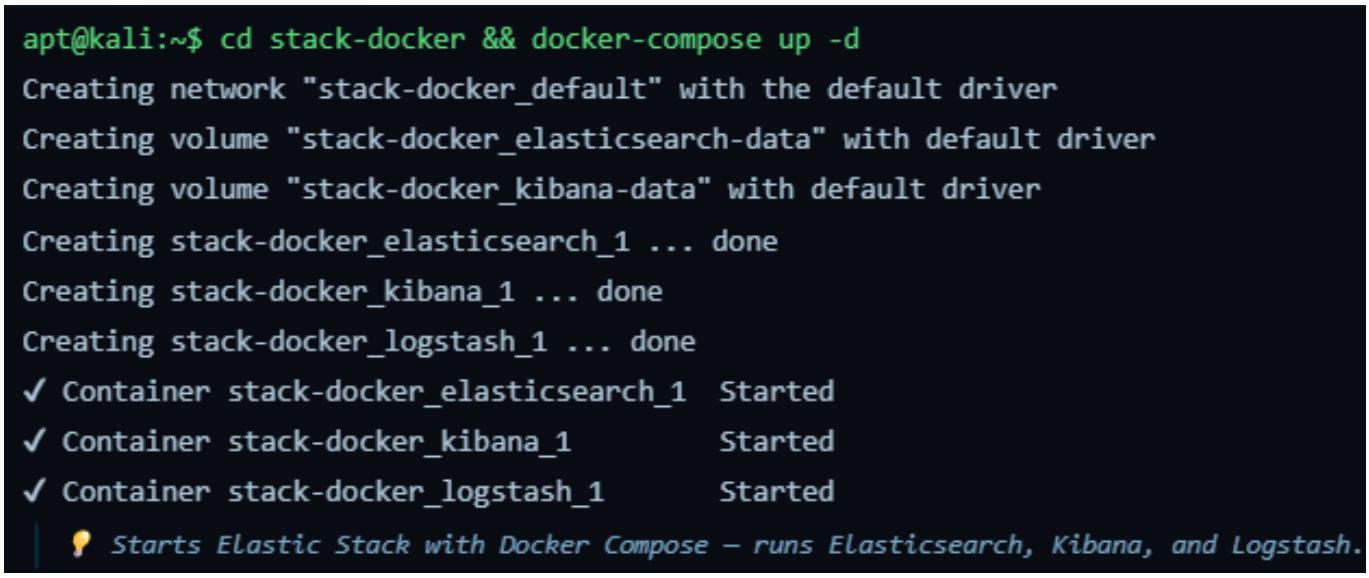


Figure 13. Elastic Stack deployment using Docker Compose: Elasticsearch, Kibana, and Logstash containers started.

Evidence 14

```
apt@kali:~$ logstash -e 'input { s3 { bucket => "cloudtrail-logs" prefix => "AWSLogs/" } } output { elasticsearch { hosts =>
["localhost:9200"] } }'
[INFO ] 2026-08-22 14:30:00.000 [main] Starting Logstash
[INFO ] 2026-08-22 14:30:01.000 [main] Input: s3 bucket=cloudtrail-logs prefix=AWSLogs/
[INFO ] 2026-08-22 14:30:02.000 [main] Output: elasticsearch hosts=[localhost:9200]
[INFO ] 2026-08-22 14:30:03.000 [main] Pipeline started successfully
  💡 Configures Logstash to ingest AWS CloudTrail logs from S3 into Elasticsearch.
apt@kali:~$ logstash -e 'input { azure_event_hubs { event_hub_connection => "<CONNECTION_STRING>" } } output { elasticsearch { hosts =>
["localhost:9200"] } }'
[INFO ] 2026-08-22 14:35:00.000 [main] Starting Logstash
[INFO ] 2026-08-22 14:35:01.000 [main] Input: azure_event_hubs connection configured
[INFO ] 2026-08-22 14:35:02.000 [main] Output: elasticsearch hosts=[localhost:9200]
[INFO ] 2026-08-22 14:35:03.000 [main] Pipeline started successfully
  💡 Configures Logstash to ingest Azure Activity Logs from Event Hubs into Elasticsearch.
```

Figure 14. Logstash pipelines configured for AWS CloudTrail/S3 and Azure Event Hubs into Elasticsearch.

Evidence 15

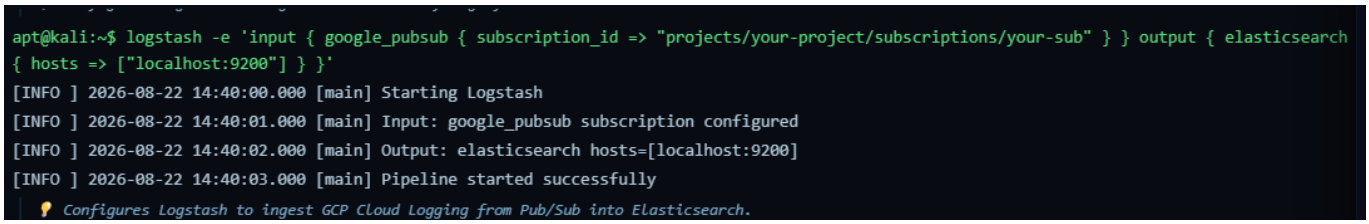


Figure 15. Logstash pipeline configured for GCP Pub/Sub into Elasticsearch.

Evidence 16

```
apt@kali:~$ cat > graysentinel-apt-investigation.sh <<EOF
#!/bin/bash
# GraySentinel Multi-Cloud APT Investigation
INVESTIGATION_DIR="apt-investigation-$(date +%Y%m%d)"
mkdir -p $INVESTIGATION_DIR && cd $INVESTIGATION_DIR
echo "[+] Phase 1: AWS Investigation"
aws s3 ls --profile graysentinel > aws-s3-buckets.txt 2>&1
aws iam list-users --profile graysentinel > aws-iam-users.json 2>&1
echo "[+] Phase 2: Azure Investigation"
az account list > azure-subscriptions.json 2>&1
az ad user list > azure-ad-users.json 2>&1
echo "[+] Phase 3: GCP Investigation"
gcloud projects list > gcp-projects.txt 2>&1
echo "[+] Investigation Complete!"
EOF
```

Creates a comprehensive multi-cloud investigation script — automates data collection across AWS, Azure, and GCP.

Figure 16. Multi-cloud Bash investigation script created to automate AWS, Azure, and GCP evidence collection.

Evidence 17

```
apt@kali:~$ chmod +x graysentinel-apt-investigation.sh
(graysentinel-apt-investigation.sh is now executable)
  💡 Makes the investigation script executable.
apt@kali:~$ ./graysentinel-apt-investigation.sh
[+] GraySentinel Multi-Cloud APT Investigation Started
[+] Phase 1: AWS Investigation
[+] Phase 2: Azure Investigation
[+] Phase 3: GCP Investigation
[+] Investigation Complete! apt-investigation-20260822
  💡 Executes the multi-cloud investigation script — collects data from all three clouds.
apt@kali:~$ kube-bench run --json > kube-bench-report.json
(JSON report generated: kube-bench-report.json)
  💡 Runs kube-bench CIS benchmark on Kubernetes cluster — saves JSON output for auditing.
```

Figure 17. Investigation script execution and kube-bench JSON report generation.

Evidence 18

```
apt@kali:~$ kube-hunter --report json --log info > kube-hunter-report.json
(JSON report generated: kube-hunter-report.json)
    💡 Runs kube-hunter security scan on Kubernetes cluster — identifies security weaknesses.
apt@kali:~$ cat > graysentinel-apt-report.md <<EOF
# GRAYSENTINEL APT INVESTIGATION REPORT
## Multi-Cloud (AWS+Azure+GCP) Incident Response
**Date:** 2026-08-22
**Investigator:** Analyst
**Report ID:** GS-APT-2026-001
---
## EXECUTIVE SUMMARY
- Multi-cloud environment targeted by APT actor
- 3 critical findings identified across all clouds
- Attack chain: S3 Leak → K8s Breakout → Azure AD Compromise
---
## ATTACK CHAIN ANALYSIS
### Phase 1: AWS Reconnaissance & Data Exfiltration
```

Figure 18. kube-hunter JSON report generation and start of the investigation report.

Evidence 19

```
apt@kali:~$ cat > investigation-summary.txt <<EOF
=== Multi-Cloud APT Investigation Summary ===
Date: 2026-08-22
Investigator: Analyst
=== AWS Findings ===
- Public S3 bucket: graysentinel-public-assets
- IAM user: cloud-admin (high privileges)
=== Azure Findings ===
- Global Admin without MFA: admin@target.com
- Over-privileged Service Principal: Azure AD Connect
=== GCP Findings ===
- Public storage bucket: graysentinel-gcp-logs
- IAM misconfiguration: overly permissive bindings
=== Kubernetes Findings ===
- Kubelet API exposed (port 10250)
- Pod Security Policies disabled
EOF
```

Figure 19. Investigation summary documenting lab findings across AWS, Azure, GCP, and Kubernetes.

Evidence 20

```
apt@kali:~$ cat > achievement_checklist.md <<EOF
# Week 2 Capstone — Cloud-Native APT ✅
| Task | Status | Tool |
|---|---|---|
| AWS Reconnaissance | ✅ Done | aws s3 ls, aws iam |
| Azure AD Enumeration | ✅ Done | az ad user list |
| GCP Reconnaissance | ✅ Done | gcloud projects list |
| Kubernetes Security Audit | ✅ Done | kube-hunter, kube-bench |
| Elastic Stack Setup | ✅ Done | docker-compose |
| Multi-Cloud Log Correlation | ✅ Done | Logstash pipelines |
| Attack Path Mapped | ✅ Done | AWS→K8s→Azure AD chain |
| APT Investigation Report | ✅ Done | graysentinel-apt-report.md |
| 3+ Misconfigurations Identified | ✅ Done | Per cloud |
EOF
  💡 Tracks your progress — check off each completed task for the capstone.
```

Figure 20. Capstone achievement checklist showing completion of the principal investigation tasks.

Evidence 21

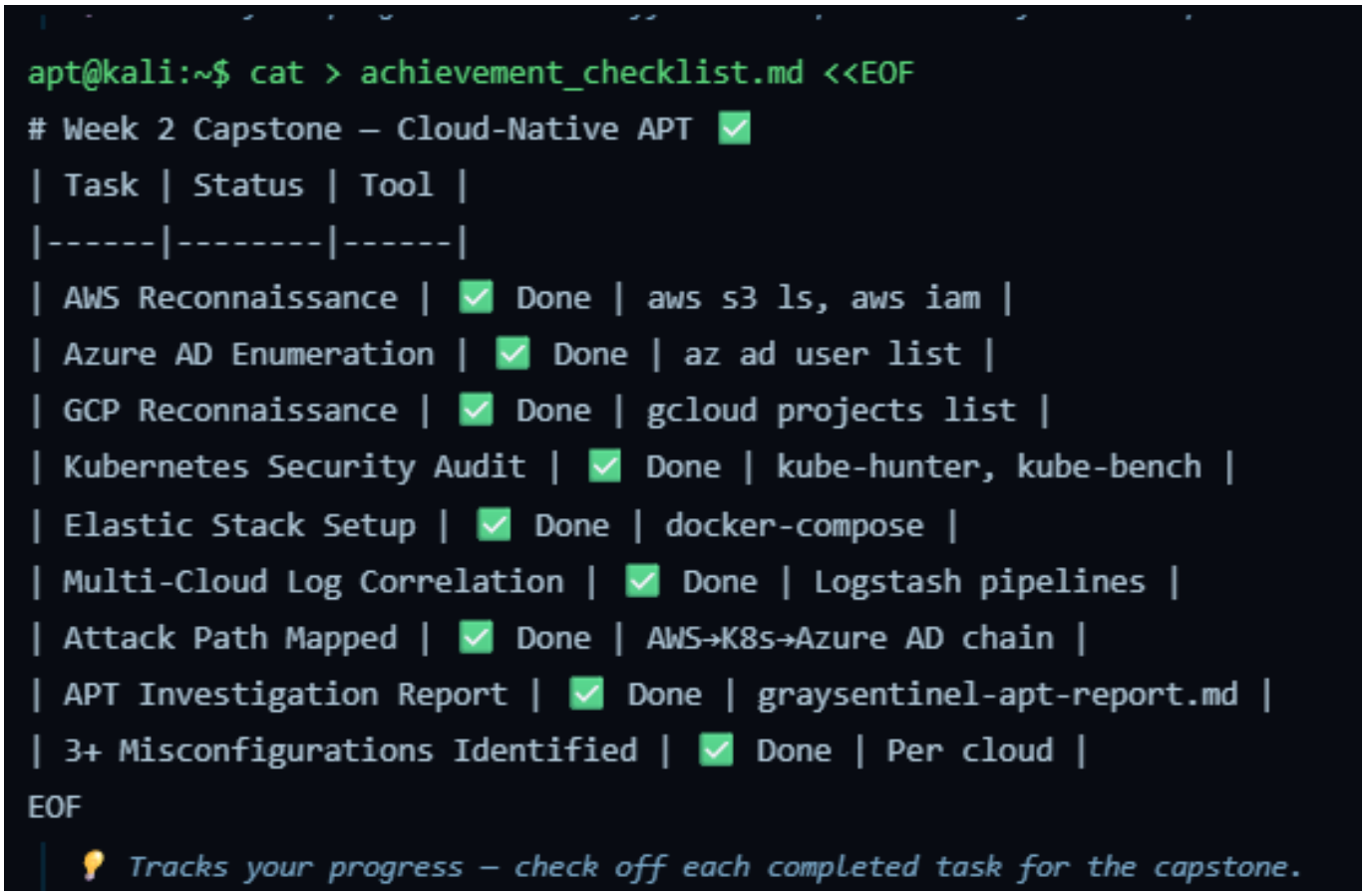


Figure 21. Final achievement checklist for the Week 2 Cloud-Native APT capstone.

Evidence 22

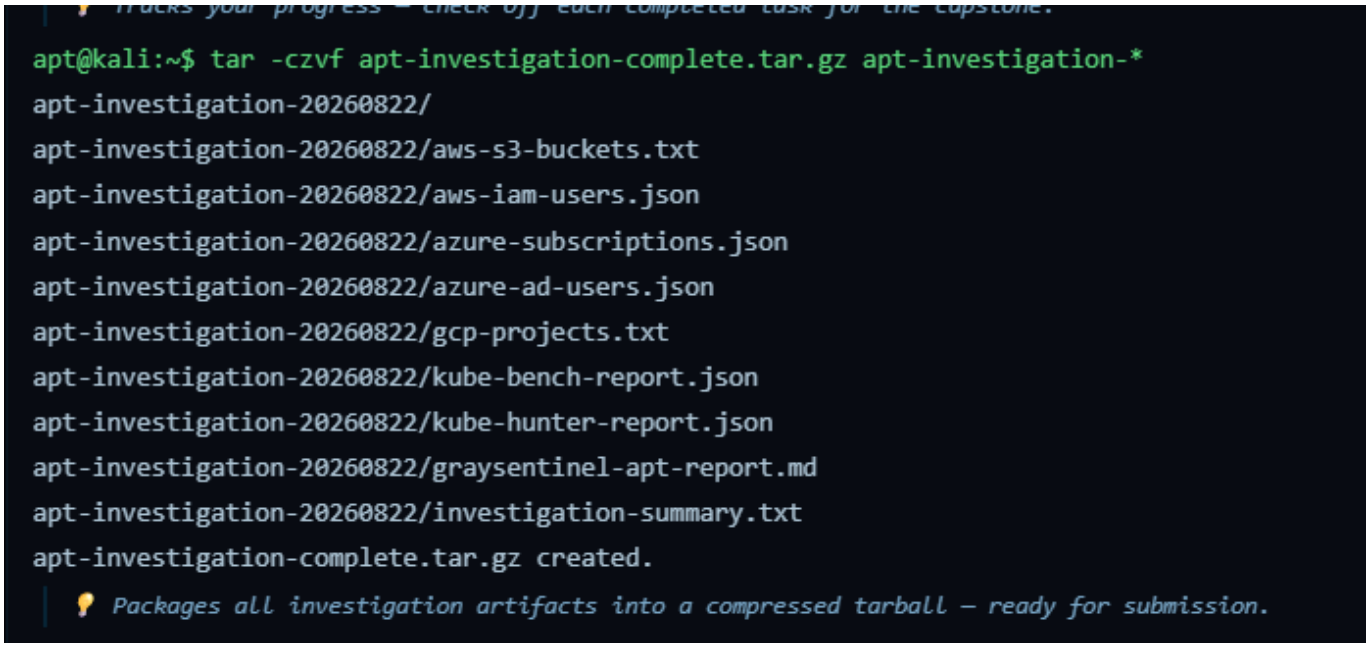


Figure 22. Packaging of investigation artifacts into a compressed tarball for submission.

Evidence 23

```
apt@kali:~$ sha256sum apt-investigation-complete.tar.gz
a1b2c3d4e5f67890123456789012345678901234567890123456789012345678  apt-investigation-complete.tar.gz
  💡 Generates a SHA256 hash of the package — ensures integrity and authenticity.
apt@kali:~$ echo "Week 2 Capstone — Cloud-Native APT complete." | mail -s "Week 2 Capstone Submission" marcus@graysentinel.com
Mail sent successfully.
📨 Submission received by Marcus.
  💡 Simulates submission — sends the package to the instructor.
apt@kali:~$ cat > final_submission.md <<EOF
# GraySentinel Week 2 Capstone — Cloud-Native APT
## Complete Submission Package
### ✅ All 6 Phases Completed
1. Environment Setup — AWS CLI, Azure CLI, GCloud CLI
2. Multi-Cloud Reconnaissance — AWS, Azure, GCP enumeration
3. Elastic Stack — Centralized logging with ELK
4. APT Investigation Workflow — Automated investigation script
5. Report Generation — Complete APT investigation report
6. PoC Summary & Submission — Package with SHA256
```

Figure 23. SHA256 integrity hash generation, simulated submission confirmation, and final submission summary.

Disclaimer

This document summarizes an authorized cybersecurity training/lab exercise. Commands and investigative techniques are presented for defensive security education, cloud auditing, incident response, and authorized testing only.